

Лекция 4. Организация защищенных каналов: OpenVPN Server

Цель лекции

Понять назначение OpenVPN для защищенного удаленного доступа к внутренним ресурсам

Различать Remote Access VPN и Site-to-Site VPN по архитектуре и маршрутизации

Объяснять роль TUN, TAP, PKI, сертификатов, TLS и клиентского профиля

Настраивать маршруты, DNS, IP forwarding, NAT и firewall для VPN-доступа

Диагностировать подключение и блокировать клиента через отзыв сертификата

Учебные вопросы

1. VPN, Remote Access и Site-to-Site; сравнение OpenVPN, WireGuard и IPsec
2. Сервер, клиент, виртуальный интерфейс и туннель; TUN и TAP
3. PKI: CA, серверные и клиентские сертификаты
4. Серверная и клиентская конфигурация; маршруты и DNS
5. IP forwarding, маршрутизация, NAT и firewall
6. Отзыв сертификата, CRL, журналы и диагностика

1. OpenVPN создает защищенный VPN-туннель

VPN (Virtual Private Network) - логический канал поверх другой сети

Не всякий туннель сам по себе дает криптографическую защиту

OpenVPN создает защищенный VPN-туннель: шифрует трафик, контролирует целостность и аутентифицирует стороны

Это отличает OpenVPN от простых туннельных технологий без шифрования

VPN не отменяет firewall, учет доступа и принцип минимально необходимого доступа

Remote Access и Site-to-Site решают разные задачи

Remote Access VPN подключает отдельного пользователя, ноутбук или администратора к сети организации

Site-to-Site VPN соединяет две сети, например головной офис и филиал

В Remote Access важны клиентские профили, индивидуальные сертификаты и политика доступа

В Site-to-Site важнее стабильная маршрутизация между подсетями

В этой лекции основной сценарий - Remote Access к внутреннему серверу web01

OpenVPN, WireGuard и IPsec выбирают под сценарий

OpenVPN подходит для Remote Access и иногда Site-to-Site, использует TLS, сертификаты, UDP или TCP

WireGuard работает через публичные ключи peer-узлов и имеет простую конфигурацию

IPsec работает на уровне IP и широко поддерживается сетевыми устройствами

OpenVPN обычно использует полноценную PKI и удобен для отзыва клиентских сертификатов

WireGuard идентифицирует peer по public key, поэтому учет ключей особенно важен

IPsec часто выбирают для интеграции

OpenVPN через UDP обычно предпочтительнее TCP

UDP 1194 - типичный транспорт OpenVPN для
обычного VPN-доступа

TCP-транспорт может помочь пройти некоторые
ограничения сети

TCP over TCP может ухудшать работу при потерях и
задержках

При потере пакетов два уровня TCP начинают
одновременно управлять повторной передачей
TCP для OpenVPN выбирают осознанно, а не как
универсально лучший вариант

Для учебного стенда используем `proto udp`

2. OpenVPN строит туннель между клиентом и сервером

OpenVPN Server принимает подключения и назначает VPN-параметры клиентам

OpenVPN Client использует профиль подключения, сертификат и закрытый ключ

Tunnel - логический защищенный канал поверх IP-сети

После подключения на клиенте и сервере появляется виртуальный интерфейс

Через этот интерфейс идут пакеты к внутренним ресурсам

Внешний адрес клиента и VPN-адрес клиента являются разными адресами

Топология: vpn-client — vpn01 — Internal LAN — web01

Remote Access VPN: защищенный доступ клиента к внутреннему серверу web01



Путь к web01 (пример)

1. Клиент подключается к `vpn01:1194/UDP`
2. Устанавливается TLS-сессия, аутентификация по сертификату
3. Создается `tun0 (10.8.0.0/24)`
4. Клиент отправляет трафик к `192.168.30.10` через туннель
5. `vpn01` маршрутизирует пакет во внутреннюю сеть
6. `web01` обрабатывает запрос и отправляет ответ обратно
7. Ответ возвращается через `vpn01` по туннелю клиенту

Адресный план

- `vpn-client` (внешний): `172.16.100.50/D4`
- `vpn01 enp0s3` (внешний): `172.16.100.10/24`
- `vpn01 enp0s8` (внутренний): `192.168.30.1/24`
- `web01` (внутренний): `192.168.30.10/24`
- VPN tunnel (`tun0`): `10.8.0.0/24`

Условные обозначения

- VPN-туннель (шифрование, аутентификация, целостность)
- Физическое соединение
- 🔒 Защищенное VPN-соединение
- 🔄 Коммутатор / внутренняя сеть



Важно: VPN не отменяет firewall, учет доступа и принцип минимально необходимого доступа.

Адресный план лабораторного стенда должен быть однозначным

vpn-client external: 172.16.100.50/24

vpn01 external enp0s3: 172.16.100.10/24

vpn01 internal enp0s8: 192.168.30.1/24

web01 internal: 192.168.30.10/24

OpenVPN tunnel tun0: 10.8.0.0/24

Корпоративные и домашние подсети клиента не должны пересекаться

Путь пакета зависит от маршрута и NAT

Клиент получает VPN IP, например 10.8.0.2

Запрос к web01 идет по туннелю: 10.8.0.2 ->
192.168.30.10

Без NAT web01 должен иметь обратный маршрут к
10.8.0.0/24 через vpn01

С NAT web01 видит источник как внутренний адрес
vpn01

Маршрутизация лучше для аудита, NAT проще для
быстрого доступа

Лаборатория должна явно выбрать одну схему и
проверять ее признаки

СХЕМА: ПУТЬ ПАКЕТА ЧЕРЕЗ OPENVPN К ВНУТРЕННЕМУ СЕРВЕРУ



- 1** Клиент формирует пакет для 192.168.10.20
Источник: 10.8.0.10 (VPN IP клиента)
- 2** Пакет шифруется и отправляется в OpenVPN-сервер
Через интернет по UDP 1194
- 3** Сервер расшифровывает пакет
Проверяет маршрут до 192.168.10.20
- 4** Пакет отправляется во внутреннюю сеть
Через LAN-интерфейс (192.168.10.1)

- 5** Внутренний сервер получает пакет
IP назначения: 192.168.10.20
- 6** Ответ от сервера к клиенту
Идёт обратно к VPN-серверу
- 7** Сервер шифрует ответ и отправляет в интернет
Клиенту на его внешний адрес
- 8** Клиент расшифровывает ответ и принимает его
Соединение установлено, обмен данными

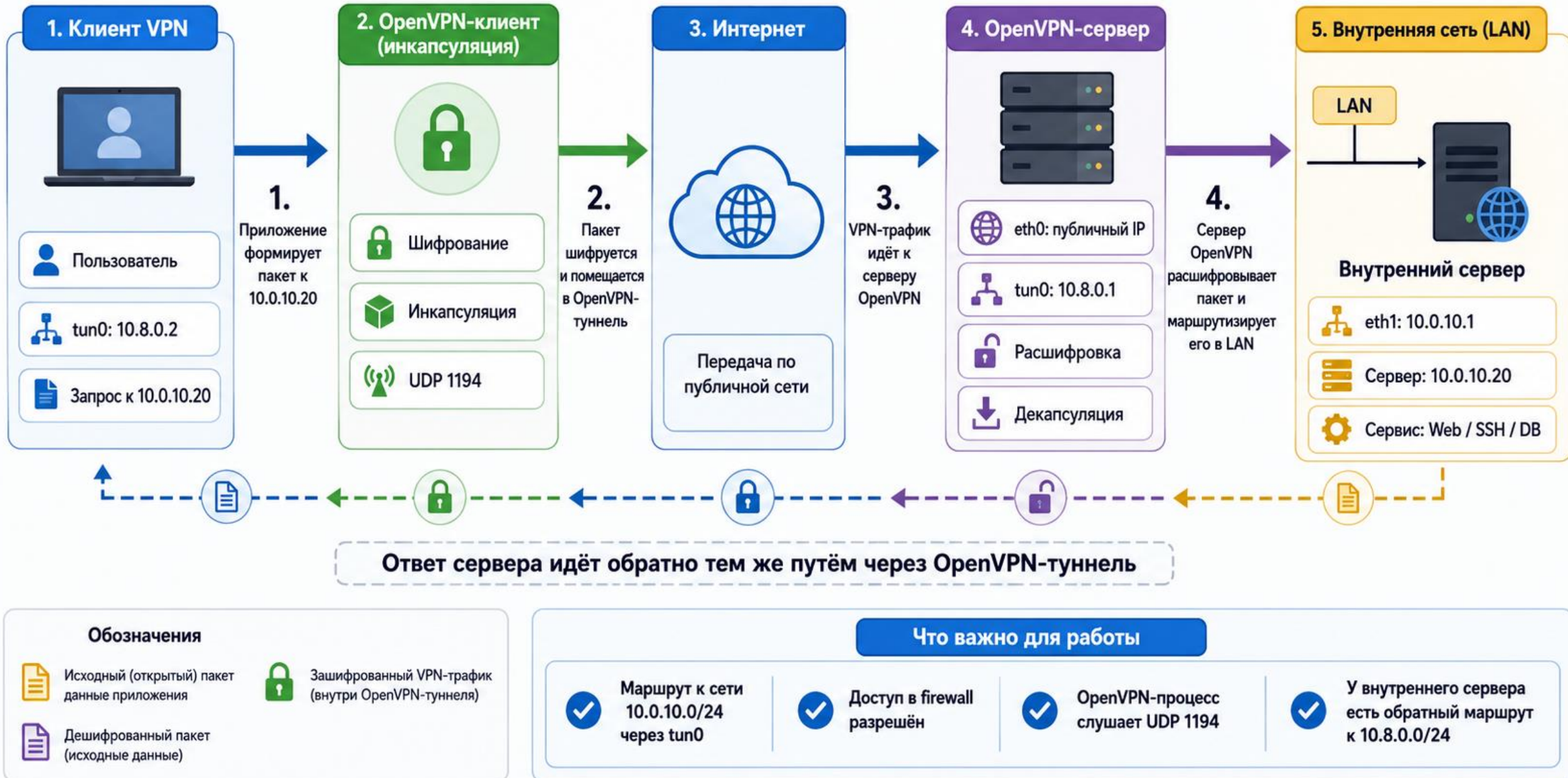
ПРИМЕР АДРЕСАЦИИ

- VPN-клиент: 10.8.0.10
- VPN-сервер (WAN): 203.0.113.5
- VPN-сервер (LAN): 192.168.10.1
- Внутренняя сеть: 192.168.10.0/24
- Внутренний сервер: 192.168.10.20

ВАЖНО

- Трафик между клиентом и сервером защищён шифрованием.
- Маршруты до внутренних сетей указываются на клиенте (push или настройка маршрутов).
- Межсетевой экран на сервере должен разрешать трафик из VPN к внутренней сети.

Схема: путь пакета через OpenVPN к внутреннему серверу



TUN и TAP работают на разных сетевых уровнях

TUN - виртуальный L3-интерфейс для маршрутизируемых IP-пакетов

TAP - виртуальный L2-интерфейс для Ethernet-кадров

TUN использует IP routing, разные подсети, push route и L3 firewall

TAP использует Ethernet bridging, один L2-домен, broadcast и ARP через VPN

Для обычного доступа к web, SSH или файловому серверу по IP достаточно TUN

TAP может хуже поддерживаться на некоторых современных и мобильных клиентах

Для Remote Access обычно выбирают TUN

TUN проще документировать, маршрутизировать и фильтровать

Клиент получает адрес из отдельной VPN-подсети, например 10.8.0.0/24

Администратор явно задает, какие внутренние сети доступны через VPN

Firewall может разрешить только нужные сервисы, например HTTPS к web01

TAP не нужен только ради доступа к серверу по IP

В учебной лаборатории dev tun уменьшает количество L2-побочных эффектов

3. PKI подтверждает подлинность сервера и клиентов

PKI (Public Key Infrastructure) - инфраструктура открытых ключей и сертификатов

CA (Certificate Authority) - центр сертификации, который подписывает сертификаты

Серверный сертификат подтверждает клиенту, что он подключается к правильному VPN-серверу

Клиентский сертификат подтверждает серверу личность клиента

CSR (Certificate Signing Request) - запрос на выпуск сертификата

Компрометация CA опасна, потому что доверие к CA

Сертификаты должны использоваться по назначению

Extended Key Usage - расширение сертификата, указывающее допустимое назначение ключа

Серверный сертификат обычно имеет назначение `serverAuth`

Клиентский сертификат обычно имеет назначение `clientAuth`

На сервере директива `remote-cert-tls client` проверяет клиентское назначение сертификата

На клиенте директива `remote-cert-tls server` проверяет серверное назначение сертификата

Без такой проверки сертификат той же СА может быть

Закрытый ключ СА лучше не хранить на VPN-сервере

VPN-серверу нужен CA certificate для проверки подписей клиентов

VPN-серверу обычно не нужен CA private key для ежедневной работы

На vpn01 хранят CA certificate, server certificate, server private key, CRL и tls-crypt key

Закрытый ключ СА желательно держать отдельно на защищенной машине выпуска сертификатов

Если vpn01 будет взломан, отдельное хранение СА затруднит выпуск новых доверенных сертификатов

Это снижает последствия компрометации VPN-шлюза

Закрытые ключи нужно защищать правами и процессом выдачи

Private Key - закрытый ключ, который должен храниться только у владельца

Пример защиты серверного ключа: `sudo chown root:root /etc/openvpn/server/server.key`

Пример прав: `sudo chmod 600 /etc/openvpn/server/server.key`

Клиентские private key нельзя отправлять обычной почтой или хранить в общей папке

Ключи не должны попадать в Git, публичный отчет или учебный скриншот

Каждому пользователю или устройству нужен

Уникальный сертификат клиента упрощает отзыв и аудит

Один общий профиль для всех сотрудников нельзя считать нормальной эксплуатационной практикой

Уникальный Common Name помогает понимать, кто подключался

Индивидуальный сертификат можно отозвать без влияния на остальных пользователей

Отдельные сертификаты позволяют создавать разные правила доступа

При компрометации одного ноутбука не нужно менять credential у всех клиентов

Директиву duplicate-cn обычно не включают без

tls-auth и tls-crypt защищают управляющий канал по-разному

tls-auth добавляет HMAC-проверку управляющих пакетов OpenVPN

tls-crypt дополнительно шифрует и аутентифицирует управляющий канал

Оба механизма помогают отбрасывать посторонние пакеты до дорогой TLS-обработки

Они не заменяют сертификаты, TLS и шифрование data channel

Для новой лаборатории логично использовать tls-crypt та. key, если версия OpenVPN это поддерживает

При tls-auth обычно указывают направление: server 0.

4. `server.conf` начинается с транспорта и TUN

`port 1194` - слушать стандартный учебный порт

OpenVPN

`proto udp` - использовать UDP как транспорт

`dev tun` - создать маршрутизируемый TUN-интерфейс

`server 10.8.0.0 255.255.255.0` - shortcut для VPN-

подсети и адресов клиентов

`topology subnet` - использовать понятную адресацию

клиентов в одной VPN-подсети

`keepalive 10 120` - проверять живость клиента и

соединения

server.conf должен ссылаться на PKI-файлы

ca /etc/openvpn/server/pki/ca.crt - доверенный CA certificate

cert /etc/openvpn/server/pki/issued/vpn01.crt - сертификат VPN-сервера

key /etc/openvpn/server/pki/private/vpn01.key - закрытый ключ VPN-сервера

dh none - для современной elliptic curve-конфигурации без отдельного DH-файла

ecdh-curve prime256v1 - пример эллиптической кривой для TLS

tls-crypt /etc/openvpn/server/ta.key - защита

server.conf задает маршруты, DNS и эксплуатационные параметры

remote-cert-tls client - проверять клиентское
назначение сертификата

push "route 192.168.30.0 255.255.255.0" - передать
клиенту маршрут к Internal LAN

push "dhcp-option DNS 192.168.30.53" - передать
клиенту внутренний DNS-сервер

persist-key и persist-tun - сохранять ключи и TUN-
интерфейс при reconnect

user nobody и group nogroup - снижать привилегии
процесса после запуска

verb 3 - умеренный уровень подробности журнала для

server 10.8.0.0 255.255.255.0 не просто задает пул

Директива server является shortcut для набора внутренних настроек OpenVPN

Она задает VPN-подсеть и адреса клиентов

Она формирует серверную сторону туннеля и связанные маршруты

Поведение адресации зависит от topology subnet или старой topology net30

Для лаборатории topology subnet проще: клиенты получают обычные адреса из 10.8.0.0/24

Если топология не совпадает с объяснением, студент неправильно прочитает маршруты

Криптографические директивы зависят от версии OpenVPN

В старых примерах часто встречаются cipher AES-256-CBC и auth SHA256

В современных версиях OpenVPN используется согласование data-ciphers

Пример: data-ciphers AES-256-GCM:AES-128-GCM:CHACHA20-POLY1305

Для совместимости иногда добавляют data-ciphers-fallback AES-256-CBC

Нельзя механически копировать старую конфигурацию без проверки версии и журналов

Перед лабораторией нужно проверить синтаксис на

ментский .ovpn-профиль содержит адрес сервера и материалы досту

client - режим клиента

dev tun и proto udp должны соответствовать серверу

remote vpn.example.test 1194 - имя или адрес VPN-

сервера до подключения туннеля

resolv-retry infinite и nobind удобны для roaming-клиента

persist-key и persist-tun сохраняют состояние при reconnect

remote-cert-tls server проверяет, что клиент подключается к серверному сертификату

.ovpn может ссылаться на отдельные файлы сертификатов

ca ca.crt - доверенный CA certificate

cert client01.crt - сертификат клиента client01

key client01.key - закрытый ключ клиента

tls-crypt ta.key - общий ключ защиты управляющего канала

verb 3 - умеренный уровень подробности журнала клиента

Все эти файлы должны быть переданы клиенту защищенным способом

Встроенный .ovpn-профиль содержит credential внутри одного файла

Встроенный профиль может содержать блоки <ca>, <cert>, <key> и <tls-crypt>

Такой файл удобен для импорта в клиентское приложение

Но внутри находится client private key

Файл .ovpn нужно защищать как полноценный credential

Нельзя прикладывать полный .ovpn с ключом в отчет или общий чат

При компрометации профиля сертификат client01
НУЖНО ОТОЗВАТЬ И ВЫПУСТИТЬ НОВЫЙ

Имя VPN-сервера должно разрешаться до подключения VPN

remote vpn.example.test 1194 используется до
появления туннеля

Клиент должен разрешить это имя через публичный
DNS, внешний лабораторный DNS или hosts

Внутренний DNS, передаваемый через VPN, не
поможет найти VPN-сервер до подключения

Для лаборатории можно использовать прямой
внешний IP vpn01

Проверка до подключения: getent hosts
vpn.example.test

Если имя VPN-сервера не разрешается. OpenVPN даже

DNS Push зависит от клиентской ОС и приложения

Сервер может передать DNS через push "dhcpcd-option DNS 192.168.30.53"

Windows OpenVPN Client обычно умеет применять DNS-настройки

Linux-клиенту может потребоваться интеграция с systemd-resolved, NetworkManager или update-resolv-conf

На мобильных клиентах DNS применяется через собственный VPN API

Проверка Linux: getent hosts web01.example.test и resolvectl status

Split Tunnel и Full Tunnel направляют разный трафик

Split Tunnel направляет через VPN только корпоративные сети

Пример split: `push "route 192.168.30.0 255.255.255.0"`

Full Tunnel направляет почти весь IPv4-трафик клиента через VPN

Пример full: `push "redirect-gateway def1"`

Split Tunnel снижает нагрузку, но Интернет клиента не контролируется VPN-шлюзом

Full Tunnel дает централизованный контроль, но требует NAT в Интернет и большей пропускной способности

5. IP Forwarding включает пересылку между интерфейсами

IP forwarding - режим, при котором Linux пересылает пакеты между интерфейсами

Без forwarding vpn01 не будет маршрутизировать трафик клиента к Internal LAN

Временная проверка: `sysctl net.ipv4.ip_forward`

Временное включение: `sudo sysctl -w`

`net.ipv4.ip_forward=1`

Постоянно лучше настраивать отдельным файлом в `/etc/sysctl.d`

После включения проверяют значение

`net.ipv4.ip_forward = 1`

Постоянный forwarding лучше документировать отдельным sysctl-файл

Команда `echo 'net.ipv4.ip_forward=1' | sudo tee /etc/sysctl.d/99-openvpn-forward.conf` создает отдельный файл настройки

Команда `sudo sysctl --system` применяет настройки из `sysctl.d`

Команда `sysctl net.ipv4.ip_forward` проверяет результат

Ожидаемый вывод: `net.ipv4.ip_forward = 1`

Отдельный файл лучше показывает, что forwarding включен именно для OpenVPN

Редактирование `/etc/sysctl.conf` допустимо, но хуже документирует назначение изменения

Корпоративный вариант предпочитает маршрутизацию без NAT

Internal gateway должен знать маршрут: 10.8.0.0/24 via 192.168.30.1

Если web01 использует этот gateway по умолчанию, достаточно маршрута на шлюзе Internal LAN web01 будет видеть реальный VPN IP клиента, например 10.8.0.2

Это упрощает аудит, журналирование и фильтрацию клиентов по адресу

Проверка обратного пути выполняется на внутреннем gateway или web01

Если обратного маршрута нет, запрос до web01

NAT является альтернативой обратному маршруту

NAT (Network Address Translation) изменяет адрес источника или назначения пакета

Masquerade на vpn01 скрывает VPN-клиента за внутренним адресом vpn01

Плюс NAT: не нужен маршрут 10.8.0.0/24 на внутренних серверах

Минус NAT: web01 видит vpn01, а не реальный адрес клиента 10.8.0.x

Для аудита и точных правил доступа маршрутизация обычно лучше NAT

В учебной лекции NAT показывается как альтернатива.

nftables Masquerade нужно ограничивать нужными сетями

Пример таблицы: `table ip nat`

Пример chain: `chain postrouting { type nat hook postrouting priority srcnat; }`

Правило: `oifname "enp0s8" ip saddr 10.8.0.0/24 ip daddr 192.168.30.0/24 masquerade`

`enp0s8` - интерфейс `vpn01` в Internal LAN

Ограничение `ip daddr 192.168.30.0/24` не дает применить `masquerade` шире лабораторной задачи

Имена интерфейсов и адреса нужно адаптировать к реальному стенду

Firewall нужно разделять на Input и Forward

Input - трафик к самому VPN-серверу vpn01

Forward - трафик через vpn01 между VPN-клиентом и Internal LAN

Разрешение UDP 1194 в Input не разрешает клиенту доступ к web01

Для web01 нужны отдельные forward rules

Минимальная политика должна разрешать только нужные сервисы

После настройки проверяют разрешенный и запрещенный сценарии

nftables Input открывает OpenVPN, но не всю сеть

Базовая идея input chain: policy drop

Правило ct state established,related ассерт пропускает ответы на уже разрешенные соединения

Правило iifname "lo" ассерт разрешает loopback

Правило iifname "enp0s3" udp dport 1194 ассерт открывает OpenVPN на внешнем интерфейсе

ICMP можно разрешить отдельно для диагностики

Этот набор не дает VPN-клиенту права ходить во внутреннюю сеть

nftables Forward разрешает только нужный доступ к web01

Forward chain также может иметь policy drop

ct state established,related accept пропускает

возвратный трафик разрешенных соединений

Правило iifname "tun0" oifname "enp0s8" ip saddr

10.8.0.0/24 ip daddr 192.168.30.10 tcp dport 443 accept

разрешает HTTPS к web01

SSH к web01 останется запрещенным, если нет

отдельного правила tcp dport 22

Stateful firewall избавляет от ручного описания

временных обратных портов

Это связывает тему VPN с принципом минимального

Перед настройкой firewall нужно понять, кто им управляет

На Linux могут одновременно присутствовать nftables, UFW, firewalld и iptables-nft

Docker и OpenVPN-скрипты тоже могут добавлять свои правила

Проверка nftables: `sudo nft list ruleset`

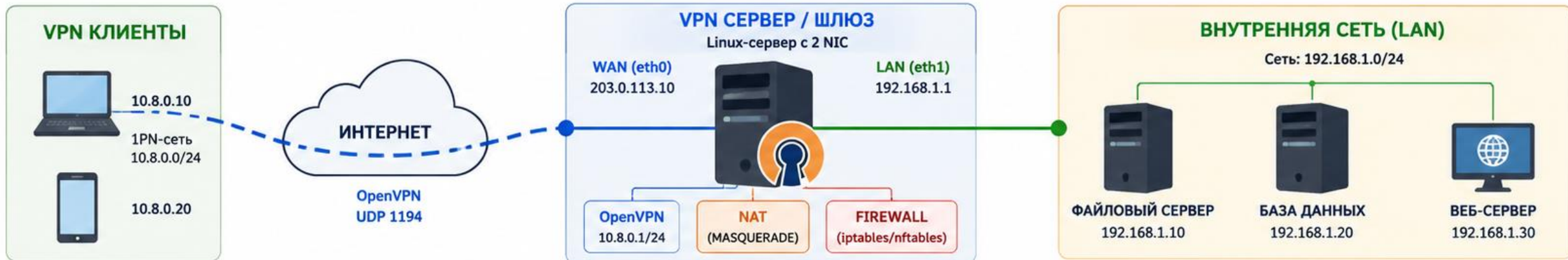
Проверка UFW: `sudo ufw status verbose`

Проверка firewalld: `sudo firewall-cmd --state` и `sudo firewall-cmd --list-all`

Нельзя без понимания одновременно менять UFW и низкоуровневый ruleset nftables

СХЕМА: МАРШРУТИЗАЦИЯ, NAT И FIREWALL ДЛЯ OPENVPN

Безопасный доступ клиентов VPN к внутренней сети



ПУТЬ ПАКЕТА



ПРАВИЛА FIREWALL (MASQUERADE GW)

- ✓ Разрешить: UDP 1194 (OpenVPN)
- ✓ Разрешить: трафик из VPN-сети 10.8.0.0/24 в LAN 192.168.1.0/24
- ✓ Разрешить: ответный трафик (ESTABLISHED, RELATED)
- ✗ Запретить: доступ из VPN в Интернет (если не требуется)
- ✗ Запретить: доступ из LAN в VPN-сеть

1. МАРШРУТИЗАЦИЯ

- Клиенты VPN получают адреса из сети 10.8.0.0/24
- На VPN-сервере включена IP-переадресация
- Маршрут к сети 10.8.0.0/24 идет через OpenVPN
- Маршрут к LAN (192.168.1.0/24) через eth1

```
sysctl -w net.ipv4.ip_forward=1
# Проверка
sysctl net.ipv4.ip_forward
# Ожидаемый вывод: net.ipv4.ip_forward = 1
```

2. NAT (MASQUERADE)

- Для доступа клиентов VPN в Интернет
- Исходящий трафик подменяет источник на внешний IP сервера (203.0.113.10)
- Для доступа в LAN NAT обычно не нужен

```
iptables -t nat -A POSTROUTING -s 10.8.0.0/24 \
-o eth0 -j MASQUERADE
# Проверка
iptables -t nat -L -n -v
```

3. FIREWALL (ФИЛЬТРАЦИЯ)

- Разрешаем только необходимый трафик
- Закрываем доступ к VPN-сети из LAN
- Применяем правило по умолчанию DROP

```
# Пример политики по умолчанию
iptables -P INPUT DROP
iptables -P FORWARD DROP
iptables -P OUTPUT ACCEPT
```

4. ИТОГ

- Клиенты VPN безопасно получают доступ к ресурсам внутренней сети
- NAT позволяет выход в Интернет (при необходимости)
- Firewall защищает сервер и внутреннюю сеть от нежелательного доступа
- Все соединения управляются политиками маршрутизации и безопасности

УСЛОВНЫЕ ОБОЗНАЧЕНИЯ

- VPN-туннель (шифрованный)
- Внутренняя сеть (LAN)
- Внешняя сеть / Интернет
- Маршрутизация
- Обратный путь (ответ)

ПРИМЕР КОНФИГУРАЦИИ OPENVPN (server.conf)

```
port 1194
proto udp
dev tun
server 10.8.0.0 255.255.255.0

push "route 192.168.1.0 255.255.255.0"
keepalive 10 120
persist-key
persist-tun
```



СЕТЕВАЯ СВОДКА

VPN-сеть: 10.8.0.0/24
LAN-сеть: 192.168.1.0/24
WAN (публичный IP): 203.0.113.10
OpenVPN порт: UDP 1194

СОВЕТ ПО БЕЗОПАСНОСТИ

- Используйте TLS-шифрование, сертификаты, сложные ключи и регулярно обновляйте систему.

6. CRL блокирует ранее выданный клиентский сертификат

CRL (Certificate Revocation List) - список отозванных сертификатов

Отзыв нужен при увольнении сотрудника или компрометации ключа

Файл CRL должен быть доступен OpenVPN-серверу для чтения

В server.conf нужна директива `crl-verify`
`/etc/openvpn/server/crl.pem`

Без `crl-verify` файл CRL может существовать, но сервер не будет использовать его при проверке клиента

После обновления CRL выполняют новое подключение

Отзыв сертификата не всегда разрывает уже активную сессию

CRL обычно проверяется при новом TLS-подключении

Если клиент уже подключен, отзыв может не отключить текущую сессию немедленно

Проверка отзыва должна включать обновление CRL на сервере

Затем существующую сессию клиента отключают или ждут ее завершения

После этого выполняют новое подключение client01

Ожидаемый результат: сервер отклоняет клиента с отозванным сертификатом

TLS-ошибки и AUTH_FAILED не нужно смешивать

TLS Verification Error обычно связан с СА, сертификатами, private key, CRL или временем Certificate Verify Failed указывает на проблему проверки сертификата

CRL-related error указывает на отзыв или недоступность списка отзыва

AUTH_FAILED чаще относится к username/password, PAM, RADIUS или внешнему authentication script

При чистой сертификатной схеме не нужно автоматически объяснять все ошибки словом

AUTH_FAILED

Диагностика OpenVPN начинается с сервера и входящего UDP

Проверить службу: `sudo systemctl status openvpn-server@server`

Проверить журнал: `sudo journalctl -u openvpn-server@server -b`

Проверить порт: `sudo ss -ltnp | grep 1194`

Проверить firewall: `sudo nft list ruleset`

Проверить входящие пакеты: `sudo tcpdump -nn -i enp0s3 udp port 1194`

Если пакеты не приходят, проблема находится до OpenVPN-процесса

После TLS проверяют туннель, маршруты и доступ к сервису

Проверить TUN на сервере: `ip -br addr show tun0`

Проверить VPN-адрес клиента: `ip -br addr`

Проверить маршрут клиента: `ip route`

Проверить forwarding: `sysctl net.ipv4.ip_forward`

Проверить обратный маршрут или NAT: `ip route get 10.8.0.2` на внутреннем gateway или web01

Проверить приложение: `curl -vk https://192.168.30.10/`

Ping не доказывает, что нужный сервис доступен

ICMP на web01 может быть запрещен, а HTTPS при этом работать

Проверять нужно реальный сервис лабораторной задачи

Команда `curl -vk https://192.168.30.10/` проверяет HTTPS-ответ

Команда `nc -vz 192.168.30.10 443` проверяет TCP-доступность порта 443

Запрещенный сценарий тоже проверяется: SSH к web01 не должен открываться

Приемка VPN должна включать разрешенный и

Конфликт локальных подсетей ломает Remote Access VPN

Домашняя сеть клиента может совпасть с корпоративной сетью

Пример конфликта: дома 192.168.30.0/24 и в организации 192.168.30.0/24

Клиент считает адрес web01 локальным и не отправляет его в туннель

Решения: изменить адресный план, применить NAT или специальную клиентскую политику

Корпоративные подсети лучше выбирать так, чтобы они реже пересекались с домашними сетями

Такой конфликт нужно проверять до обвинения

Client-Specific Configuration помогает задавать индивидуальную политику

CCD (Client-Specific Configuration) - индивидуальные настройки OpenVPN для конкретного клиента

Директива `client-config-dir /etc/openvpn/server/ccd` указывает каталог CCD

Файл `/etc/openvpn/server/ccd/client01` применяется к клиенту с Common Name `client01`

Через CCD можно задавать индивидуальные маршруты или параметры доступа

Подробная настройка CCD выходит за базовую лабораторию, но логически связана с уникальными сертификатами

риемка лаборатории должна проверять разрешенное и запрещенно

До VPN у клиента нет tun0 и маршрута к
192.168.30.0/24

После VPN клиент получает 10.8.0.x и маршрут к
192.168.30.0/24

HTTPS к web01 становится доступен: curl -vk
https://192.168.30.10/

SSH к web01 остается запрещен, если политика
доступа настроена правильно

DNS-имя web01 разрешается только если DNS push
применился клиентской ОС

Подключение с отозванным сертификатом

ДИАГРАММА: ДИАГНОСТИКА OPENVPN-ПОДКЛЮЧЕНИЯ ПО УРОВНЯМ



🎯 Принцип: проверяем **снизу вверх** — от сети до приложений

УРОВЕНЬ	ЧТО ПРОВЕРЯЕМ	КАК ПРОВЕРИТЬ (ПРИМЕРЫ)	ОЖИДАЕМЫЙ РЕЗУЛЬТАТ	ЕСЛИ НЕ РАБОТАЕТ	ПОРЯДОК ДИАГНОСТИКИ
6 ПРИЛОЖЕНИЯ (СЕРВИСЫ)	Доступ к внутренним ресурсам и сервисам через VPN	<code>ping 10.8.0.10</code> <code>telnet 10.8.0.10 3389</code> <code>curl http://10.8.0.10</code> <code>traceroute 10.8.0.10</code>	✅ Ресурсы доступны, сервисы отвечают, маршрут до хоста проходит через VPN-интерфейс	⚠️ Проверить сервисы на сервере Проверить межсетевой экран (FW) Проверить маршрутизацию на сервере Проверить правила доступа	6 Доступны ли внутренние ресурсы?
5 VPN-ТУННЕЛЬ (OPENVPN)	Состояние OpenVPN-клиента и туннеля, получение IP-адреса, обмен пакетами	<code>openvpn --status</code> <code>ip addr show tun0</code> <code>route grep 10.8.0.0</code> <code>logread grep -i openvpn</code>	✅ Клиент подключён (CONNECTED) IP-адрес назначен Трафик идёт через tun-интерфейс Нет ошибок в логах	⚠️ Проверить конфиг клиента Проверить сертификаты и ключи Проверить логи OpenVPN Проверить время на клиентах	5 Поднялся ли VPN-туннель?
4 ХОСТ / ОС (ЛОКАЛЬНЫЙ УРОВЕНЬ)	Интерфейсы, маршруты, DNS, фаервол хоста	<code>ip addr</code> <code>ip route</code> <code>resolvectl status / cat /etc/resolv.conf</code> <code>iptables -L / nft list ruleset</code>	✅ Интерфейс tun0 UP Маршруты на сети VPN есть DNS работает Фаервол не блокирует трафик	⚠️ Добавить/исправить маршруты Исправить DNS Открыть нужные порты/правила FW	4 Всё ли в порядке на хосте?
3 ТРАНСПОРТ (TCP / UDP)	Доступность порта OpenVPN (по умолчанию UDP 1194) до VPN-сервера	<code>nc -zv <vpn-server> 1194 -u</code> <code>telnet <vpn-server> 1194</code> <code>nmap -sU -p 1194 <vpn-server></code>	✅ Порт 1194/UDP открыт и доступен Ответ от сервера есть (или порт отображается как open)	⚠️ Проверить фаервол/фаервол провайдера Проверить порт и протокол в конфиге Проверить NAT/Port Forwarding	3 Доступен ли порт OpenVPN?
2 СЕТЬ (IP) (МАРШРУТИЗАЦИЯ)	Маршрутизация до VPN-сервера, доступность по IP-адресу	<code>ping <vpn-server-ip></code> <code>traceroute <vpn-server-ip></code> <code>ip route get <vpn-server-ip></code>	✅ Есть маршрут до сервера Пакеты доходят (ответ есть) Нет потерь на пути	⚠️ Проверить маршрут по умолчанию Проверить промежуточные роутеры Проверить доступ в интернет	2 Есть ли маршрут до VPN-сервера?
1 ФИЗИЧЕСКИЙ УРОВЕНЬ (L1)	Физическое подключение и базовая связь с сетью	<code>ip link show</code> <code>ethtool eth0</code> <code>ping 8.8.8.8</code>	✅ Интерфейс UP, линк активен Есть связь с внешней сетью Нет ошибок линка	⚠️ Проверить кабель/Wi-Fi Проверить драйвер/адаптер Обратиться к провайдеру	1 Есть ли физическая связь?

📋 Исправляйте проблему на текущем уровне, затем поднимайтесь выше

БЫСТРЫЕ ПРОВЕРКИ

`openvpn --status`
Быстрый статус клиента

`ip addr show tun0`
Проверка IP и интерфейса

`route | grep 10.8.0.0`
Проверка маршрута через VPN

`logread | grep -i openvpn`
Ошибки и причина разрыва

ЧАСТЫЕ ПРИЧИНЫ ПРОБЛЕМ

- Неверный конфиг (IP, порт, протокол)
- Истёкшие сертификаты / неверное время
- Блокировка портов фаерволом / провайдером
- Отсутствие маршрута до VPN-сети

ПОЛЕЗНО ЗНАТЬ

Логи OpenVPN – главный источник информации. Всегда начинайте с них.

Диагностика OpenVPN-подключения по уровням

Проверяйте сверху вниз — остановитесь на уровне, где найдено несоответствие.

УРОВЕНЬ		ЧТО ПРОВЕРЯЕМ	КАК ПРОВЕРИТЬ (примеры)	ОЖИДАЕМЫЙ РЕЗУЛЬТАТ	ЕСЛИ НЕ РАБОТАЕТ
L1		Сеть и доступность VPN-сервера - IP-адрес и маршрут - Порт UDP/TCP 1194	<pre>ping <vpn_server_ip> traceroute <vpn_server_ip> nc -vz -u <vpn_server_ip> 1194 # или: telnet <vpn_server_ip> 1194</pre>	 Узел доступен. Маршрут проходит. Порт 1194 открыт.	 Проблема с сетью/ маршрутизацией. Порт закрыт фаерволом или провайдером.
L2		Сервис OpenVPN на сервере - Процесс запущен - Слушает нужный порт	<pre>systemctl status openvpn-server@server ss -ltnp grep 1194 journalctl -u openvpn-server@server -n 50</pre>	 Сервис active (running). Слушает UDP/TCP 1194.	 Сервис не запущен или слушает другой адрес/порт.
L3		TLS-рукопожатие и шифрование - Сертификаты CA/Server - Версия TLS, шифры	<pre>openssl s_client -connect <vpn_server_ip>:1194 -tls1_2 # или: -tls1_3 / -udp (если поддерживается) # на клиенте: openvpn --verb 5 --connect-retry 2</pre>	 TLS handshake успешен. Нет ошибок сертификатов. Договорились алгоритмы.	 Ошибки сертификатов, TLS handshake failed, неверные параметры шифрования.
L4		Аутентификация клиента (сертификат/логин) - Проверка сертификата - Права клиента (CN)	<pre># на сервере: grep -E "AUTH VERIFY CN=" /var/log/openvpn.log # на клиенте в конце лога: Initialization Sequence Completed</pre>	 Client 'CN' authenticated или Initialization Sequence Completed	 AUTH_FAILED, VERIFY ERROR, клиент не допущен.
L5		VPN-интерфейс и IP - Интерфейс tun0/tap0 - IP-адрес из пула VPN - Маршруты на клиенте	<pre># клиент: ip addr show tun0 ip route show # сервер: ip addr show tun0</pre>	 tun0 up. IP из пула (напр. 10.8.0.x). Маршруты установлены.	 Нет IP на tun0. Маршрут к внутренней сети не установлен.
L6		Маршрутизация, NAT и firewall на сервере - IP forwarding - Правила firewall - NAT (при доступе в Internet)	<pre>sysctl net.ipv4.ip_forward iptables -L -n -v iptables -t nat -L -n -v</pre>	 net.ipv4.ip_forward = 1 Правила разрешают трафик. NAT работает (если нужен выход в Internet).	 Трафик запрещён firewall. Нет маршрута к сети. IP forwarding выключен.
L7		Доступ к внутреннему ресурсу (web01) - Маршрут до 192.168.30.10 - Сервис на хосте	<pre>ping 192.168.30.10 curl http://192.168.30.10 nc -vz 192.168.30.10 80</pre>	 Пинг/HTTP успешны. Сервис отвечает.	 Хост недоступен, сервис не запущен, блокирует firewall хоста.

БЫСТРЫЕ ДЕЙСТВИЯ



Нет соединения с сервером

→ Проверьте сеть, маршрут и порт 1194 (UDP/TCP)



Сервис не слушает

→ Запустите сервис OpenVPN и проверьте порт



Ошибка TLS/сертификатов

→ Проверьте CA, сертификаты, даты и имена (CN/SAN)



AUTH_FAILED

→ Проверьте клиентский сертификат и права (CN)



Нет IP на tun0

→ Проверьте пул адресов и маршруты на клиенте



Не проходит трафик

→ Разрешите forwarding и правила firewall, при необходимости NAT



Ресурс недоступен

→ Проверьте маршрут к сети и запущенный сервис



Совет: включите подробные логи на клиенте: `openvpn --verb 5` и на сервере: `verb 4-5`

УСПЕШНОЕ ПОДКЛЮЧЕНИЕ:



VPN-туннель
(10.8.0.0/24)



vpn01
(OpenVPN)



Internal LAN
(192.168.30.0/24)



web01
(192.168.30.10)

Обозначения:

-  Проверка пройдена
-  Требуется внимания
-  Проблема найдена

Итоги лекции

OpenVPN защищает трафик через TLS, сертификаты и шифрование, но не заменяет firewall

Remote Access требует клиентского профиля, уникального сертификата и понятной политики доступа

TUN проще для маршрутизируемого доступа к внутренним IP-сервисам

Рабочий VPN требует согласования `server.conf`, `.ovpn`, `forwarding`, маршрутов, DNS и firewall

CRL работает только если сервер настроен ее проверять и клиент выполняет новое подключение

Контрольные вопросы

Чем OpenVPN отличается от туннеля, который не выполняет шифрование?

Чем Remote Access VPN отличается от Site-to-Site VPN?

Почему для лаборатории выбран TUN, а не TAP?

Зачем нужны serverAuth, clientAuth и remote-cert-tls?

Почему CA private key лучше не хранить на VPN-сервере?

Чем Split Tunnel отличается от Full Tunnel?

Почему разрешение UDP 1194 в Input не дает доступа к web01?

Как проверить, что CRL действительно блокирует